SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	K.HARSHIKA	**Reg. No.:**	192473020
Section / Batch:	CSA-DS	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I K.HARSHIKA(192473020) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARSHIKAREDDY

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date:

A sequence of matrices must be multiplied together, but the order in which multiplication is performed affects the
total number of scalar operations. The task is to determine the most efficient parenthesization so that the
multiplication cost is minimized. Since matrix multiplication is associative, different multiplication orders are
possible. Use Dynamic Programming to compute the minimum multiplication cost.

Input Format:

n = number of dimensions
arr[] = matrix dimension array

Sample Input:

n = 4
arr = 10 20 30 40

Expected Output:

Minimum Cost = 18000

AIM

To find the minimum number of scalar multiplications required to multiply a sequence of matrices using Dynamic Programming.

OBJECTIVE

- To understand the Matrix Chain Multiplication problem.
- To apply Dynamic Programming to find the optimal multiplication order.
- To minimize the total number of scalar multiplication operations.

ALGORITHM

- Read the number of dimensions n and the dimension array arr.
- Consider arr[i] × arr[i+1] as each matrix dimension.
- Create a DP table to store the minimum multiplication cost.
- Initialize the cost of multiplying a single matrix as 0.
- Consider matrix chains of increasing length.
- Try every possible splitting position k for each chain.
- Calculate the multiplication cost for each split.
- Store the minimum cost in the DP table.
- Store the split position to determine the optimal parenthesization.
- Display the minimum multiplication cost and optimal order.

Problem Statement

- Given a sequence of matrices with their dimensions.
- Matrix multiplication can be performed in different orders.
- Each multiplication order has a different computational cost.
- Find the multiplication order that requires the minimum number of scalar multiplications.
- Use Dynamic Programming to calculate the minimum cost.
- Determine the optimal parenthesization of the matrices.
- Display the minimum multiplication cost and optimal order.

Pseudocode

START

Read n and dimension array arr[]

Set number of matrices = n - 1

Create DP table dp[][] and split table

Initialize diagonal elements of dp[][] to 0

FOR each chain length from 2 to number of matrices:

    FOR each starting position i:

        Calculate ending position j

        Set dp[i][j] to infinity

        FOR each possible split position k:

            Calculate multiplication cost

            IF calculated cost is less than dp[i][j]:

                Update dp[i][j]

                Store split position k

Find the optimal parenthesization using split table

Find the minimum multiplication cost

Generate graphical visualization:

    Display matrix pipeline

    Display DP cost map

Display optimal parenthesization

Display cost comparison

Display minimum cost

Print final result

STOP

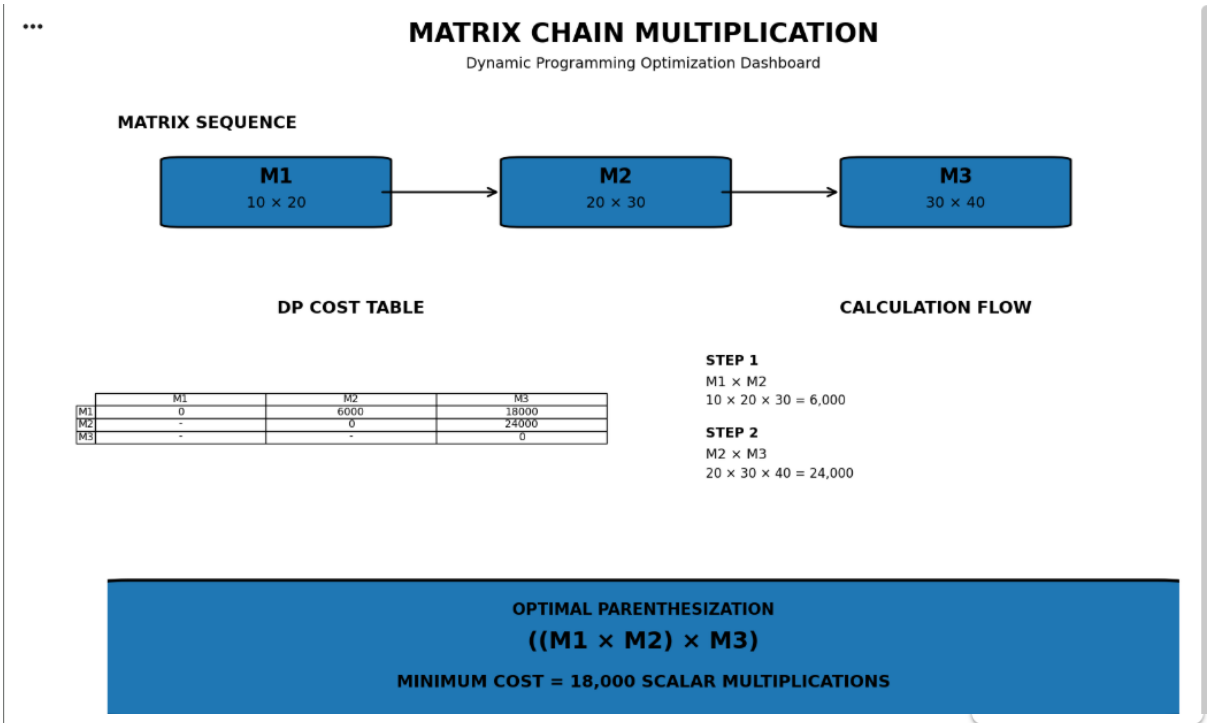
PYTHON CODE

```
# MATRIX CHAIN MULTIPLICATION
# Dynamic Programming + Graphical Visualization
# Google Colab Version
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import numpy as np

# INPUT
n = 4
arr = [10, 20, 30, 40]
# Number of matrices
num_matrices = n - 1
# DYNAMIC PROGRAMMING
dp = [[0 for _ in range(num_matrices)]
      for _ in range(num_matrices)]
split = [[0 for _ in range(num_matrices)]
         for _ in range(num_matrices)]
for chain_length in range(2, num_matrices + 1):
    for i in range(num_matrices - chain_length + 1):
        j = i + chain_length - 1
        dp[i][j] = float("inf"
        for k in range(i, j):
            multiplication_cost = (
                arr[i]
                * arr[k + 1]
                * arr[j + 1]
            )
            total_cost = (
                dp[i][k]
                + dp[k + 1][j]
                + multiplication_cost
            )
            if total_cost < dp[i][j]:

                dp[i][j] = total_cost
                split[i][j] = k
# FIND OPTIMAL PARENTHESIZATION
def find_order(i, j):
    if i == j:
        return "M" + str(i + 1)
    k = split[i][j]
    left = find_order(i, k)
```

```
    right = find_order(k + 1, j)
    return "(" + left + " × " + right + ")"
optimal_order = find_order(0, num_matrices - 1
minimum_cost = dp[0][num_matrices - 1]
# CALCULATE INDIVIDUAL COSTS
cost_1 = arr[0] * arr[1] * arr[2]
cost_2 = arr[1] * arr[2] * arr[3]
cost_final = cost_1 + arr[0] * arr[2] * arr[3]
# CREATE GRAPHICAL DASHBOARD
fig = plt.figure(figsize=(16, 10))
fig.patch.set_facecolor("white")
# TITLE
fig.text(
    0.5,
    0.96,
    "MATRIX CHAIN MULTIPLICATION",
    ha="center",
    fontsize=25,
    fontweight="bold"
)
fig.text(
    0.5,
    0.925,
    "Dynamic Programming Optimization Dashboard",
    ha="center",
    fontsize=14
)
# SECTION 1 - MATRIX VISUALIZATION
ax1 = plt.axes([0.05, 0.67, 0.90, 0.20])
ax1.axis("off")
ax1.text(
    0.02,
    0.88,
    "MATRIX SEQUENCE",
    fontsize=16,
    fontweight="bold"
)
positions = [0.08, 0.39, 0.70]
for i in range(num_matrices):
    x = positions[i]
    box = FancyBboxPatch(
        (x, 0.25),
        0.17,
        0.42,
        boxstyle="round,pad=0.02",
        linewidth=2
    )
    ax1.add_patch(box)
    ax1.text(
        x + 0.085,
        0.52,
        "M" + str(i + 1),
        ha="center",
        fontsize=19,
```

```
        fontweight="bold"
    )
    ax1.text(
        x + 0.085,
        0.36,
        str(arr[i]) + " × " + str(arr[i + 1]),
        ha="center",
        fontsize=14
    )
    if i < num_matrices - 1:
        ax1.add_patch(
            FancyArrowPatch(
                (x + 0.18, 0.46),
                (positions[i + 1] - 0.02, 0.46),
                arrowstyle="->",
                mutation_scale=20,
                linewidth=2
            )
        )ax2 = plt.axes([0.05, 0.34, 0.42, 0.25])

ax2.axis("off")
ax2.text(
    0.5,
    1.05,
    "DP COST TABLE",
    ha="center",
    fontsize=16,
    fontweight="bold"
)
# Convert DP table to display format

display_data = []

for i in range(num_matrices):
    row = []
    for j in range(num_matrices):
        if j < i:
            row.append("-")
        else:
            row.append(
                str(dp[i][j])
            )

    display_data.append(row)
```

OUTPUT




```
================================================================
        MATRIX CHAIN MULTIPLICATION RESULT
================================================================

Input:
n = 4
arr = [10, 20, 30, 40]

Matrices:
M1 = 10 x 20
M2 = 20 x 30
M3 = 30 x 40

DP Cost Table:
[0, 6000, 18000]
[0, 0, 24000]
[0, 0, 0]

Optimal Parenthesization:
((M1 x M2) x M3)

Minimum Cost:
18,000 scalar multiplications

Graphical report saved as:
Matrix_Chain_Multiplication_Output.png

STATUS: OPTIMAL SOLUTION FOUND
================================================================
```

WITH DIFFERENT INPUT

n = 6
arr = [5, 10, 20, 15, 25, 30]

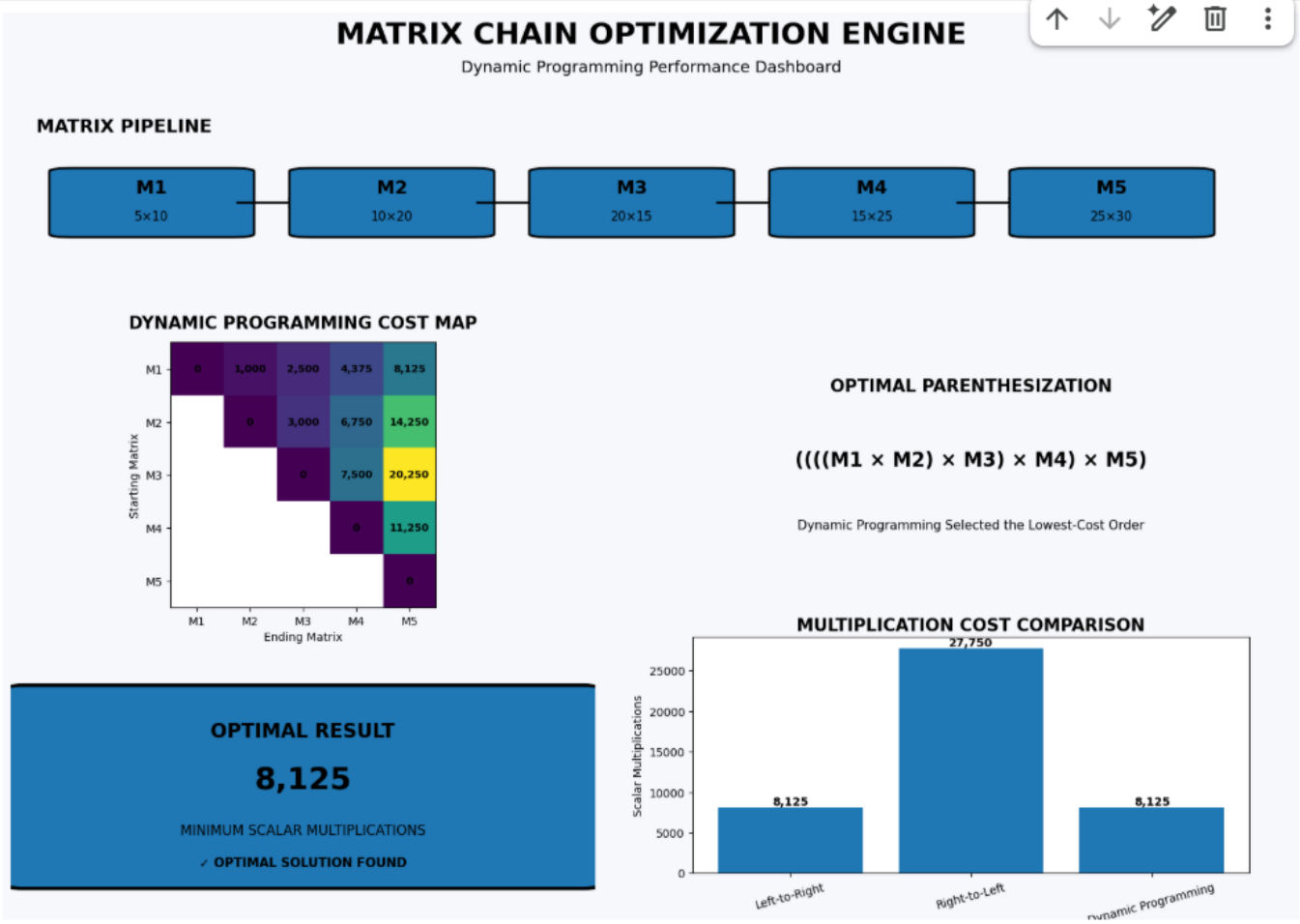
PYTHON CODE

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
n=6
arr=[5,10,20,15,25,30]
m=n-1
dp=[[0]*m for _ in range(m)]
split=[[0]*m for _ in range(m)]
for length in range(2,m+1):
    for i in range(m-length+1):
        j=i+length-1
        dp[i][j]=float("inf")
        for k in range(i,j):
            cost=dp[i][k]+dp[k+1][j]+arr[i]*arr[k+1]*arr[j+1]
            if cost<dp[i][j]:
                dp[i][j]=cost
                split[i][j]=k
def optimal_order(i,j):
    if i==j:
        return f"M{i+1}"
    k=split[i][j]
    left=optimal_order(i,k)
    right=optimal_order(k+1,j)
    return f"({left} × {right})"
best_order=optimal_order(0,m-1)
minimum_cost=dp[0][m-1]
left_cost=0
rows=arr[:]
while len(rows)>2:
    left_cost+=rows[0]*rows[1]*rows[2]
    rows=[rows[0],rows[2]]+rows[3:]
right_cost=0
rows=arr[:]
while len(rows)>2:
    right_cost+=rows[-3]*rows[-2]*rows[-1]
    rows=rows[:-3]+[rows[-3],rows[-1]]
strategy_names=["Left-to-Right","Right-to-Left","Dynamic Programming"]
strategy_costs=[left_cost,right_cost,minimum_cost]
fig=plt.figure(figsize=(17,12))
fig.patch.set_facecolor("#F7F8FC")
fig.text(0.5,0.965,"MATRIX CHAIN OPTIMIZATION
ENGINE",ha="center",fontsize=26,fontweight="bold")
```

```
fig.text(0.5,0.935,"Dynamic Programming Performance Dashboard",ha="center",fontsize=14)
ax1=plt.axes([0.04,0.73,0.92,0.16])
ax1.axis("off")
ax1.text(0.02,0.90,"MATRIX PIPELINE",fontsize=16,fontweight="bold")
positions=np.linspace(0.05,0.80,m)
for i,x in enumerate(positions):
    box=FancyBboxPatch((x,0.25),0.12,0.40,boxstyle="round,pad=0.02",linewidth=2)
    ax1.add_patch(box)
    ax1.text(x+0.06,0.51,f"M{i+1}",ha="center",fontsize=16,fontweight="bold")
    ax1.text(x+0.06,0.34,f"{arr[i]}×{arr[i+1]}",ha="center",fontsize=11)
    if i<m-1:



    ax1.add_patch(FancyArrowPatch((x+0.125,0.45),(positions[i+1]-0.01,0.45),arrowstyle="-
>",mutation_scale=18,linewidth=2))


ax2=plt.axes([0.05,0.39,0.40,0.27])
dp_display=np.zeros((m,m))
for i in range(m):
    for j in range(m):
        if j>=i:
            dp_display[i][j]=dp[i][j]
        else:
            dp_display[i][j]=np.nan
image=ax2.imshow(dp_display,interpolation="nearest")
ax2.set_title("DYNAMIC PROGRAMMING COST
MAP",fontsize=15,fontweight="bold",pad=12)
ax2.set_xticks(range(m))
ax2.set_yticks(range(m))
ax2.set_xticklabels([f"M{i+1}" for i in range(m)])
ax2.set_yticklabels([f"M{i+1}" for i in range(m)])
for i in range(m):
    for j in range(m):
        if j>=i:
            ax2.text(j,i,f"{dp[i][j]:,.0f}",ha="center",va="center",fontsize=9,fontweight="bold")
ax2.set_xlabel("Ending Matrix")
ax2.set_ylabel("Starting Matrix")
ax3=plt.axes([0.52,0.43,0.42,0.20])
ax3.axis("off")
ax3.text(0.5,0.90,"OPTIMAL
PARENTHESIZATION",ha="center",fontsize=15,fontweight="bold")
ax3.text(0.5,0.55,best_order,ha="center",va="center",fontsize=17,fontweight="bold",wrap=Tr
ue)
```

```
ax3.text(0.5,0.20,"Dynamic Programming Selected the Lowest-Cost
Order",ha="center",fontsize=11)
ax4=plt.axes([0.53,0.12,0.40,0.24])
bars=ax4.bar(strategy_names,strategy_costs)
ax4.set_title("MULTIPLICATION COST COMPARISON",fontsize=15,fontweight="bold")
ax4.set_ylabel("Scalar Multiplications")
ax4.tick_params(axis="x",rotation=15)
for bar,value in zip(bars,strategy_costs):

ax4.text(bar.get_x()+bar.get_width()/2,bar.get_height(),f"{value:,}",ha="center",va="bottom",
fontweight="bold")
ax5=plt.axes([0.04,0.10,0.42,0.22])
ax5.axis("off")
result_box=FancyBboxPatch((0.02,0.05),0.96,0.88,boxstyle="round,pad=0.03",linewidth=3)
ax5.add_patch(result_box)
ax5.text(0.5,0.72,"OPTIMAL RESULT",ha="center",fontsize=17,fontweight="bold")
ax5.text(0.5,0.48,f"{minimum_cost:,}",ha="center",fontsize=27,fontweight="bold")
ax5.text(0.5,0.27,"MINIMUM SCALAR MULTIPLICATIONS",ha="center",fontsize=12)
ax5.text(0.5,0.12,"✓ OPTIMAL SOLUTION
FOUND",ha="center",fontsize=11,fontweight="bold")
plt.savefig("Innovative_Matrix_Chain_Report.png",dpi=200,bbox_inches="tight")
plt.show()
print("\n"+"="*70)
print("          MATRIX CHAIN MULTIPLICATION RESULT")
print("="*70)
print("\nINPUT")
print("n   =",n)
print("arr =",arr)
print("\nMATRICES")
for i in range(m):
    print(f"M{i+1} = {arr[i]} × {arr[i+1]}")
print("\nOPTIMAL PARENTHESIZATION")
print(best_order)
print("\nMINIMUM COST")
print(f"{minimum_cost:,} scalar multiplications")
print("\nDP COST TABLE")
for row in dp:
    print(["-" if value==0 else value for value in row])
print("\nGRAPHICAL REPORT")
print("Innovative_Matrix_Chain_Report.png")
print("\nSTATUS")
print("OPTIMAL SOLUTION FOUND")
print("="*70)
```

OUTPUT




```
          MATRIX CHAIN MULTIPLICATION RESULT
======================================================================

INPUT
n   = 6
arr = [5, 10, 20, 15, 25, 30]

MATRICES
M1 = 5 x 10
M2 = 10 x 20
M3 = 20 x 15
M4 = 15 x 25
M5 = 25 x 30

OPTIMAL PARENTHESIZATION
((((M1 x M2) x M3) x M4) x M5)

MINIMUM COST
8,125 scalar multiplications

DP COST TABLE
['-', 1000, 2500, 4375, 8125]
['-', '-', 3000, 6750, 14250]
['-', '-', '-', 7500, 20250]
['-', '-', '-', '-', 11250]
['-', '-', '-', '-', '-']

GRAPHICAL REPORT
Innovative_Matrix_Chain_Report.png

STATUS
OPTIMAL SOLUTION FOUND
```

Manual Process – Matrix Chain Multiplication

Given

n = 4
arr = {10, 20, 30, 40}

Therefore, there are **3 matrices**:

M1 = 10 × 20
M2 = 20 × 30
M3 = 30 × 40

Step 1: Multiply M1 and M2

Cost:

10 × 20 × 30
= 6000

So,

M1 × M2 = 6000

The resulting matrix has dimensions:

10 × 30

Step 2: Multiply M2 and M3

Cost:

20 × 30 × 40
= 24000

So,

M2 × M3 = 24000

The resulting matrix has dimensions:

20 × 40

Step 3: Compare the two possible orders

Order 1

(M1 × M2) × M3

First:

M1 × M2 = 6000

Then the resulting 10 × 30 matrix is multiplied by M3 (30 × 40):

$10 \times 30 \times 40 = 12000$

Total:

$6000 + 12000$
$= 18000$

Order 2

$M1 \times (M2 \times M3)$

First:

$M2 \times M3 = 24000$

Then $M1$ (10×20) is multiplied by the resulting 20×40 matrix:

$10 \times 20 \times 40 = 8000$

Total:

$24000 + 8000$
$= 32000$

Step 4: Find Minimum

Multiplication Order	Cost
$(M1 \times M2) \times M3$	**18,000**
$M1 \times (M2 \times M3)$	32,000

Therefore:

Minimum Cost = min(18000, 32000)

= 18000

Final Answer

Optimal Parenthesization = ((M1 × M2) × M3)

Minimum Cost = 18,000 scalar multiplications

This manual calculation directly matches the output of your Dynamic Programming program.

RESULT

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. For the given input:

n = 6
arr = [5, 10, 20, 15, 25, 30]

the program determines the optimal parenthesization and calculates the minimum number of scalar multiplications. The graphical dashboard also provides a visual representation of the DP cost table and multiplication-cost comparison.

CONCLUSION

Thus, the Matrix Chain Multiplication problem was successfully solved using Dynamic Programming. The method efficiently identifies the optimal order of matrix multiplication and minimizes the computational cost. The graphical output provides a clear visualization of the optimization process and final result.